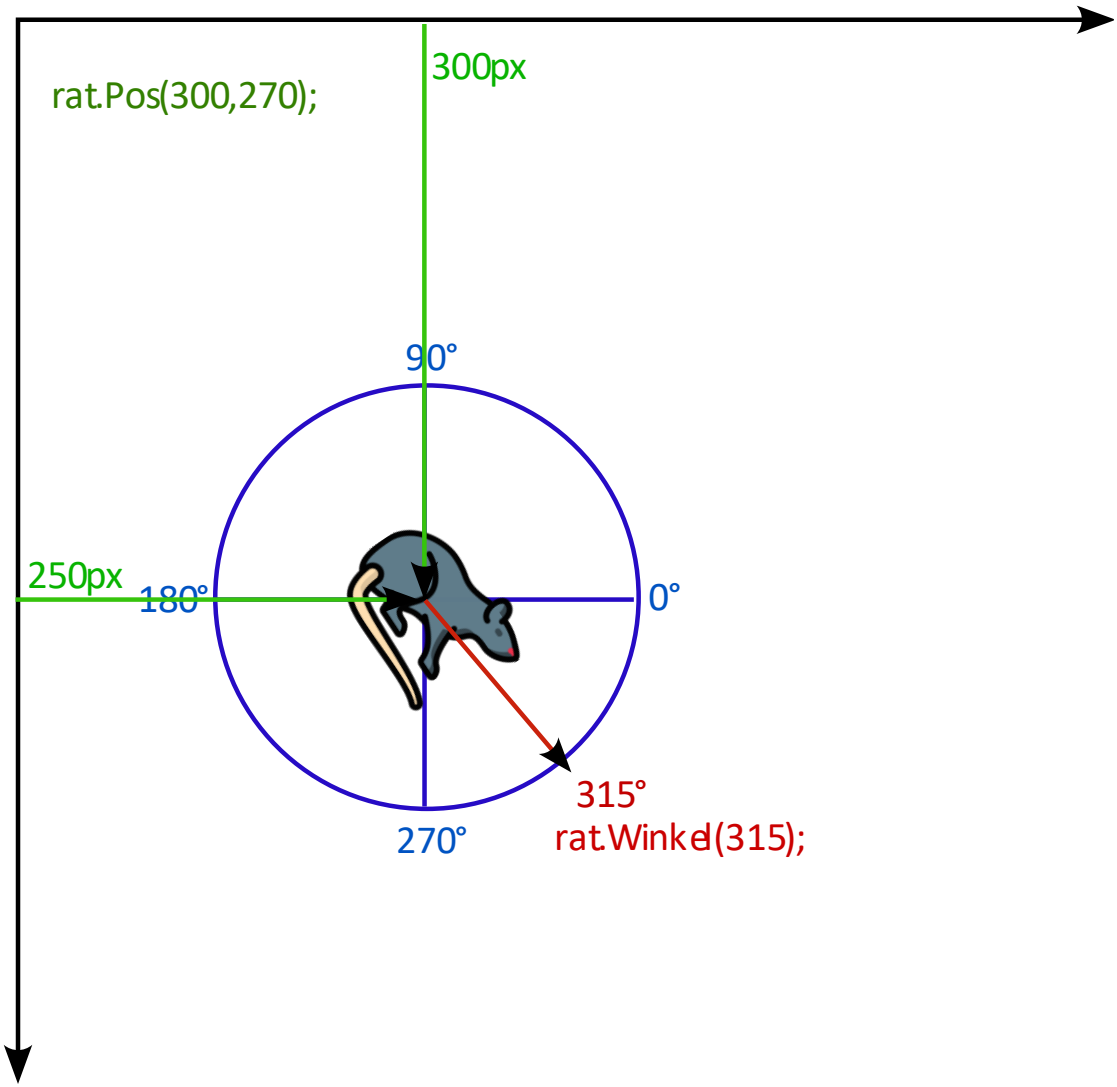
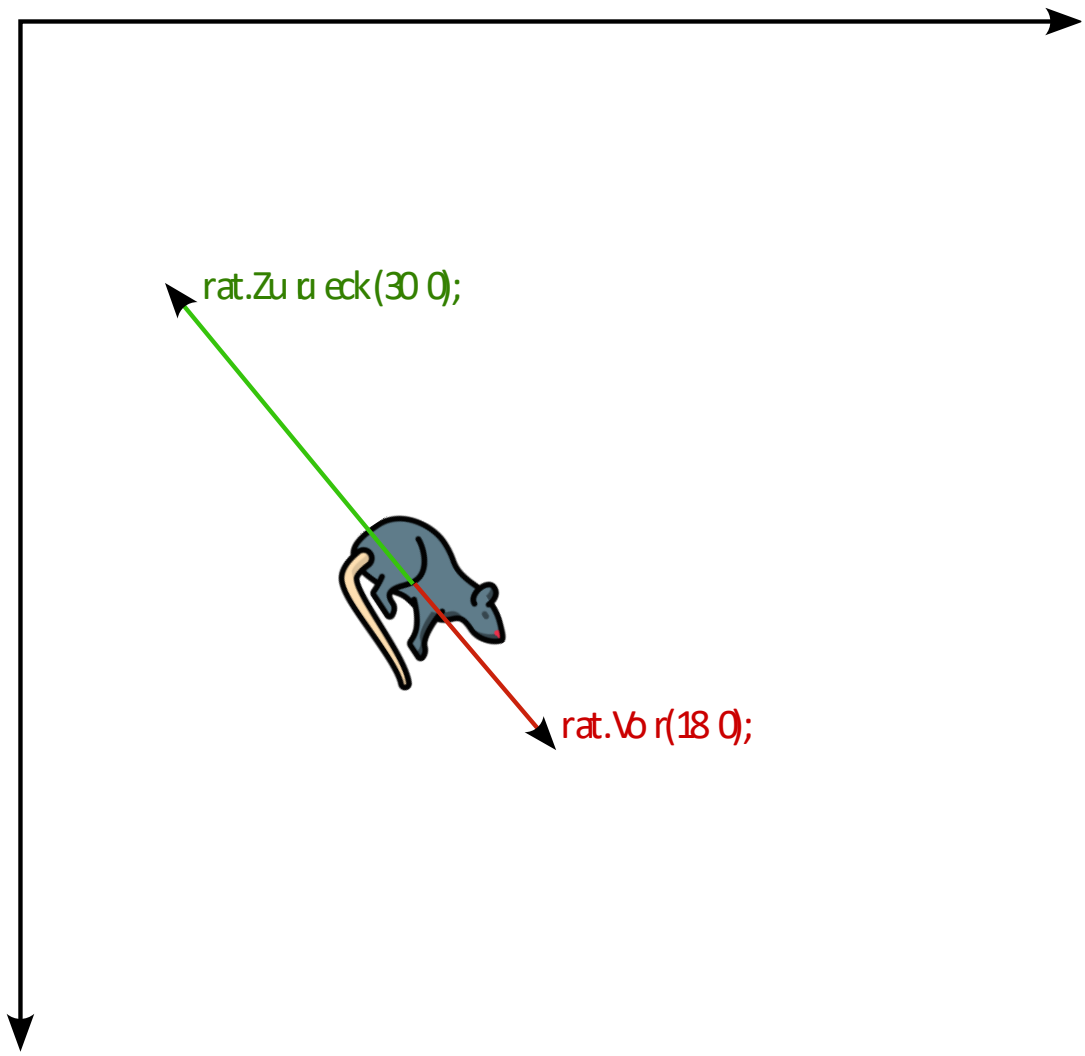
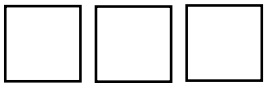
```
public Vector2 VektorKoordinate(double magnitude, double angleDegrees)

public Task BisHier()

public void Schnell()

public void Winkel(double grad)

public void Pos(double x, double y)

public void Vor(double distanz)

public void Zurueck(double distanz)

public void DreheRechts(double grad)

public void DreheLinks(double grad)

public void SpurLoeschen()
```





Aufgaben:

1. 1. Zeichne ein Quadrat
2. Zeichne 3 Quadrate nebeneinander



3. Zeichne 3 Quadrate nebeneinander, mit 5 Pixel Abstand. Tipp: Bedenke, dass die Breite des Quadrates etwas mit der Position des „nächsten Quadrates“ zu tun hat!

Methoden

Oftmals willst du eine Gruppe von Anweisungen mehrfach an verschiedenen Stellen in deinem Programm nutzen. Dann lohnt es sich, eine Methode dafür zu schreiben. In deinem Labor sind schon einige Methoden definiert.

Plenum:

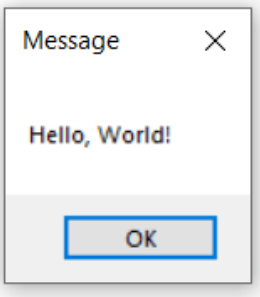
Welche Methoden gibt es in deinem Programm?

Woran erkennst du, dass eine neue Methode im Programm beginnt?

Welche Anweisungen gehören „zu“ der jeweiligen Methode?

```
public class test
{
    public void Main()
    {
        EinfachesDemo(19);

        int Antwort = DemoMitRueckgabe(7,5);
        MessageBox.Show(Antwort);
    }
    public void EinfachesDemo(int Zahl)
    {
        MessageBox.Show(Zahl);
    }

    public int DemoMitRueckgabe(int Zahl, int NochNeZahl)
    {
        int Ergebnis = Zahl + NochNeZahl;
        MessageBox.Show(Ergebnis);
        return Ergebnis;
    }
}
```

Zu Beginn startet die Methode „Main“. Sie ruft dann „EinfachesDemo“ auf. Der Methode wird ein Int „übergeben“. Schau dir an, wie so eine Übergabe funktioniert. Was passiert dann in der Methode? Was gibt die MessageBox aus? Rechts siehst du ein Beispiel für eine MessageBox, damit du weißt, was mit MessageBox gemeint ist. Die gibt natürlich etwas ganz anderes aus.



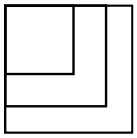
Klicken oder tippen Sie hier, um Text einzugeben.

Dann startet die Methode „DemoMitRueckgabe. Ihr werden sogar zwei Int-Werte übergeben. Diese Methode gibt am Ende auch einen Int-Wert „zurück“. Was geben die MessageBoxen aus? Wie – mit welchem Befehl - funktioniert die Rückgabe? Woran erkennst du, dass ein Int-Wert zurückgegeben wird?

Klicken oder tippen Sie hier, um Text einzugeben.

Aufgaben:

1. Schreibe selber eine Methode „public void Quadrat(int Kantenlänge). Die soll – wie du erraten kannst – ein Quadrat mit der übergebenen Kantenlänge zeichnen.
2. Erweitere deine Methode um eine Position (Point pos). Dieser soll die Ratte am Anfang an genau diese Position setzen, so dass das Quadrat da gezeichnet wird.
3. Zeichne 3 unterschiedlich große Quadrate ineinander. Was verändert sich ? Führe eine Variable für die Veränderung ein.
4. Zeichne nochmal die 3 Quadrate aus Aufgabe 2. Nutze deine neue Methode. Wie hat sich dein Programm „vereinfacht“?



Schleifen

Schleifen stellen eine weitere Möglichkeit dar, Anweisungen mehrmals abzuarbeiten. Generell gibt es zwei Ausgangslagen: Entweder du weißt, wie oft etwas abgearbeitet werden soll oder eben nicht. Wenn du ein Quadrat zeichnen willst, dass nun mal aus 4 gleichen Seiten besteht, weißt du das vorher. Wenn du eine Tabelle mit Namen erstellst und da immer neue Namen eingegeben werden können, bis jemand auf „Fertig“ klickt, weißt du es halt nicht. Wir fangen mit der Variante an, wo du zu Beginn weißt, wie oft die Schleife durchlaufen werden soll.

```
public class Schleifen
{
    public void main()
    {
        for(int i = 0; i < 10; i++)
        {
            Console.Writeline(i);
        }
    }

}
```

Die for-Schleife wirkt zu Beginn etwas „Sperrig“ in manchen andeen Sprachen schreibt man sie so: „for(n = 0 to 10). Dann zählt sie eben von 0 – 10. In C# (und Java) ist die Schreibweise etwas anders, aber das ermöglicht dir später tolle Erweiterungen.

Plenum:

Überlege selbst: Wo wird bei der C#-for-Schleife ein Anfangswert gesetzt?

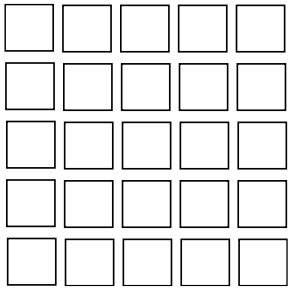
Wo steht ein Endwert?

Was soll dieses i++?

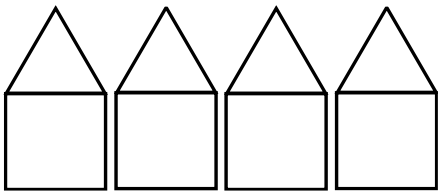
Aufgaben

1. Nutze eine for-Schleife um deine Quadrat-Methode zu verkürzen. Überleg: Wie oft musst du „Vor“ und „DreheLinks“ ausführen? Mach dafür eine for-Schleife und schreib die Kommandos für Vor und dreheLinks 1x da rein!

2. Zeichne nun 10 Quadrate nebeneinander. (Tipp: Du kannst auch in einer for-Schleife Methoden starten!)
3. Zeichne ein Quadratgitter: 5 Nebeneinander, 5 Reihen untereinander. Tipp: Du kannst in einer for-Schleife eine weitere for-Schleife erzeugen. Überleg, wie beide abgearbeitet werden! Tipp: Erst denken – dann schaffen. Ohne Plan, wie die Schleifen abgearbeitet werden, schaffst du es nicht!

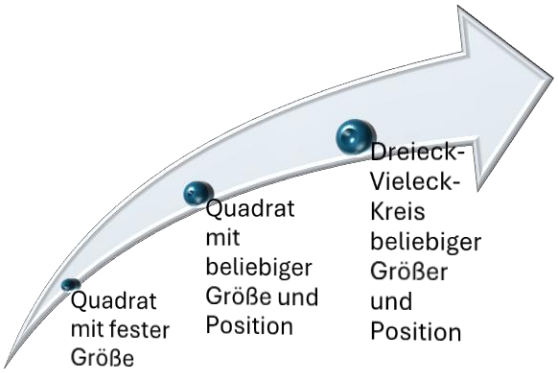


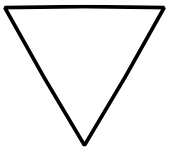
4. Schreibe eine Methode „public void Dreieck(int groesse, Point pos)“. Du kannst es dir schon denken: Die soll ein Dreieck zeichnen und genauso wie die Quadratmethode funktionieren. Wichtig: Die Spitze des Dreieck soll nach oben zeigen (hat was mit DreheLinks/DreheRechts zu tun)!
5. Zeichne aus einem Quadrat und einem Dreieck ein Haus.
6. Schreibe eine Methode um ein Haus zu zeichnen. Tipp: Du kannst in einer Methode andere Methoden aufrufen!
7. Zeichne eine Reihe von Häusern! Wenn deine Haus-Methode noch keinen Parameter für die Position hat: Jetzt wäre der Zeitpunkt einen „einzubauen“, sonst wird’s komplex!



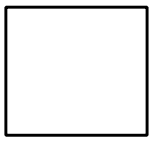
Parametrisieren

In der Informatik entwickeln wir möglichst allgemeingültige Lösungen. Das bedeutet: Wir schreiben unsere Programme so, dass wir nachher möglichst flexibel verschiedene Lösungen umsetzen können. Du hast schon damit begonnen: Deine Quadratmethode kann schon jetzt Quadrate in unterschiedlicher Größe an beliebige Stellen zeichnen. Dazu hast du Zahlenwerte, die du anfangs „fest“ in deinem Programm eingegeben hattest, zu Variablen gemacht. Man nennt den Vorgang „Parametrisieren“, weil du jetzt Parameter angeben kannst. Als nächstes sollst du eine Methode schreiben, die eine Figur mit beliebig vielen Ecken zeichnen kann. Dazu diese Vorüberlegung:

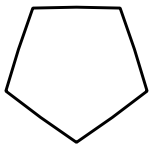




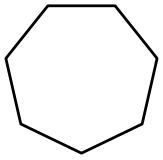
360°/3 = 120°



360°/4=90°



360°/5=72°



360°/____= _____°???

360°/_____ _____ = _____ _____

Aufgabe

Schreibe eine eigene Methode public void **Vieleck(int Eckenzahl, int Kantenlänge).** Die Bilder links helfen dir, herauszufinden, wie du den Winkel der Ecken bestimmen kannst. Überlege auch, ob du für den Winkel immer „glatte Zahlen“ herausbekommst oder vielleicht auch Kommazahlen. Kommazahlen werden in C# als double bezeichnet, glatte als int.

Fazit:

Um eine public void VielEck(int Eckenzahl, int Kantenlänge) zu schreiben kann ich nicht mehr einen festen Winkel in meinem Quellcode brauchen (90° oder 120°).

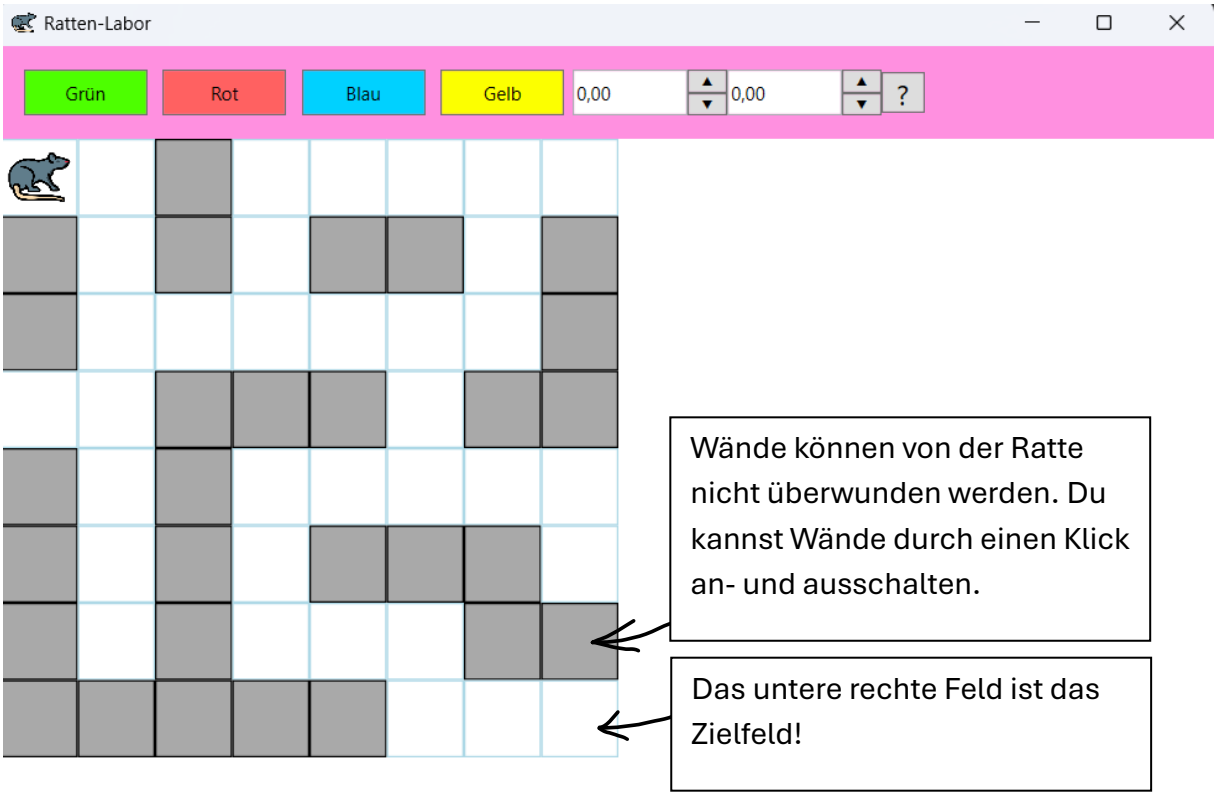
Ich brauche eine Variable. Was genau? Klicken oder tippen Sie hier, um Text einzugeben.

Wenn ich den Vollwinkel gleichmäßig auf die Anzahl der Ecken auf**teilen** will, muss ich das wie folgt berechnen:Klicken oder tippen Sie hier, um Text einzugeben.

Die Anzahl der Ecken und die Länge der Kanten übergebe ich als Klicken oder tippen Sie hier, um Text einzugeben.

Labyrinth-Challenge

Ziel der Challenge ist es in allen drei Tiers, dass die Ratte das Zielfeld erreicht. Dieses ist immer das Feld ganz unten rechts. Ihr dürft Teams zu maximal 3 Entwicklern bilden.



Bronze-Tier-Challenge:

Du erstellst sowohl die Methode als auch das Labyrinth. Dadurch weißt du von Anfang an genau, wie du deine Ratte steuern musst. Deine Methode muss also die Ratte auf einem vorher schon bekannten Weg durch das Labyrinth lenken.

Silver-Tier-Challenge:

Du erstellst die Methode, die für beliebige Labyrinthe funktioniert. Diese Labyrinthe haben aber keine Sackgassen oder Rundwege. Es gibt also immer einen eindeutigen Weg vom Start zum Ziel, ohne Abzweigungen.

Gold-Tier-Challenge:

Du erstellst die Methode, die für beliebige Labyrinthe ohne Einschränkung funktioniert. Die Labyrinthe dürfen also auch Sackgassen und Rundwege enthalten.

Neue Wirkweise der Befehle

Der **Vor()-**Befehl: Dieser Befehl funktioniert etwas anders als du ihn von früher kennst: Er prüft, ob vor der Ratte ein Weg oder eine Wand ist. Wenn ein Weg ist, bewegt er die Ratte ein Kästchen vor. Wenn ein Hindernis da ist, bewegt er die Ratte nicht. Außerdem – und das ist wichtig – gibt er dir einen Boolwert zurück: True, wenn die Ratte sich bewegt hat und false, wenn ein Hindernis im Weg ist. Das kannst du nutzen, um zu testen, ob der Weg vor dir frei ist. Da die Ratte immer nur ein Feld vorrückt, gibst du in der Klammer keinen Parameter mehr an!

DreheLinks() und **DreheRechts():** Sie drehen die Ratte immer im rechten Winkel. Wenn du die Ratte also 180° drehen willst, musst du zweimal die Methode aufrufen. Auch hier gibst du keinen Parameter an!

Winkel(Richtung): Hier ändert sich der Parameter: Früher konntest du hier eine Gradzahl angeben. Jetzt kannst du nur noch Werte von 0-3 angeben:

Parameter	Richtung
0	Rechts
1	Hoch
2	Links
3	Runter

KI-Nutzung

Wichtiger Hinweis: Wie auch vorher darfst du KI nutzen, um die Methoden zu erstellen. Um die Challenge abzuschließen, musst du den **Code am Smartboard vorstellen und erläutern, wie er funktioniert.** Wenn du dann zwar funktionstüchtigen Code vorzeigst, aber keine Ahnung hast **warum** er funktioniert, gilt das leider nicht. Tipp: Die KI erklärt dir geduldig, warum etwas so läuft. Und auch ich beantworte dir **konkrete** Fragen zu deinem Code…